OPEN SOURCE · DETERMINISTIC · BUILT FOR AI AGENTS

A seatbelt for the AI acting on your systems.

AI agents can now write to your database, edit your store, send your emails. That's wonderful — until the day one of them gets it catastrophically wrong. Reeflex sits at the door of your resource and checks every write **before** it happens. Not *"is this user allowed?"* but **"is this action safe, given the impact it would actually have?"** Deterministic, logged, and honest.

> **We love open source — so the core is free, forever.**
>
> The engine, the specification, the policy language, the reference adapters — all Apache-2.0, yours to run, fork, and build on. No lock-in, no metering on the decision path, no asterisks. Safety infrastructure should belong to everyone who needs it. We'd rather earn your trust than trap it.

Every action is a crossroad. Turn left, it runs; turn right, it's stopped, or held for whoever you designate to resolve it — human, agent, or automation. Reeflex decides which way — on **calculated impact**, not identity or guesswork — and records **who decided, what the impact was, and who accepted the risk.**

WordPress / WooCommerce · Databases (SQL) · GraphQL · Website chatbots · Generic REST APIs

THE DECISION MODEL

A decision is a risk, accepted at a crossroad

Every write an agent attempts is a fork in the road. Reeflex answers three questions at that fork — deterministically, with no LLM in the decision path — and turns the answer into an auditable record.

Who dictates?

The **policy** decides — not the agent, not the model. A versioned rule pack (Rego) owned by you. The same input always yields the same verdict.

What is the impact?

Computed, not declared. Reeflex measures the action on three axes and its magnitude — how many rows, how reversible, how far it reaches.

Who bears the risk?

Low impact runs unattended. High impact is **held** — resolved by the principal you designate (human, agent, or automation) under your resolution policy — and that decision is logged as evidence.

The Action Envelope — how any action is measured

Each adapter normalizes a raw action into one universal shape. Three axes plus a magnitude are enough to reason about danger, independent of technology:

Axis	Values (least → most severe)
reversibility	`reversible · recoverable · irreversible`
blast_radius	`single · scoped · broad · systemic`
externality	`internal · outbound · physical`
magnitude	`count — how many entities affected`

Missing or non-canonical values are coerced to the most-restrictive member — fail-closed by construction. A typo never becomes a silent allow.

The verdict is one of three. The policy chooses by explicit precedence (`deny > require_approval > allow`), so exactly one verdict is produced:

`allow` Impact within budget — executes normally.

`require_approval` Dangerous but legitimate — held for your designated principal to resolve.

`deny` Catastrophic — blocked even if the caller is an admin.

Every verdict carries the **reason** and the **rule** that fired — the raw material of a compliance record.

Fragmentation resistance. An agent cannot slip past a threshold by splitting one big action into many small ones. A per-session cumulative ledger sums impact across the whole session, so “delete 5 rows, one hundred times” trips the same budget as “delete 500 rows.”

POLICY ENGINE — YOUR RULES, NOT OURS

The engine is deterministic. The policy is yours.

Reeflex decides through **Open Policy Agent (OPA)** — the same policy engine used at Netflix, Goldman Sachs, and across the CNCF ecosystem. Rules are written in **Rego** and live in a policy directory the engine reads at evaluation time. That single design choice answers the two questions every serious buyer asks: **can we write our own rules, and can we bring our internal security policy?** Yes to both — by construction, not by promise.

Custom policies — built in

- Rules are `.rego` files in a directory (`REEFLEX_POLICY_DIR`). Add or change a rule without touching the engine.
- The engine evaluates one query — `data.reeflex.policy.decision` — so any rule set that answers it plugs straight in.
- Policy is **versioned like code**: reviewed, diffed, rolled back, and provable for any date (audit).
- **No LLM anywhere** in the path — same envelope + same policy → same decision, every time.

> **The default pack (R1–R5)** ships as a starting point: read-only allow, irreversible-broad-prod → approval, irreversible-systemic-prod → deny, and the session delete budget. Treat it as a baseline to extend, not a fixed ruleset.

Each rule is a decades-old safety principle — change control, egress control, transaction thresholds, fraud-detection velocity checks — applied to agent actions.

Clients bring their internal policy

A regulated entity already has written security rules. Reeflex doesn't replace them — it **executes** them at the resource boundary. They author their own Rego (or you translate their policy into it), drop it in their policy directory, and the engine enforces it.

Example: a bank's own rule

```
# no bulk customer export, ever, outbound
decision := { "decision": "deny", … } if {
  input.action.verb == "read"
  input.axes.externality == "outbound"
  input.target.entity == "customer_pii"
  input.magnitude.count > 50
}
```

Their rule, their thresholds, their jurisdiction — enforced deterministically, logged as evidence.

Where the money is. The core engine and the base policy pack are open. Paid value is the **curated policy packs** per vertical (WooCommerce, Postgres, GDPR/NIS2 compliance) — maintained, updated, and mapped to regulation — plus the managed distribution that keeps every client's rules fresh. You own your rules; we keep the expert rules current.

Not a broad authorization engine. OPA can answer "is this identity allowed?" Reeflex uses OPA to answer a narrower, harder question — "given the computed impact, is this *safe*?" Same engine, deliberately pointed at impact rather than identity.

HOW IMPACT IS COMPUTED

Not a magic score — a deterministic reading of the action

"Computed impact" isn't an AI guess or a mysterious risk number. It is a plain, reproducible classification of what the action *is* and how much it *touches*, done in three layers. The same action always reads the same way.

1 Per-action classification — in the adapter

The adapter reads the raw action and derives the three axes plus a magnitude from **explicit rules**, not opinion. A `DROP TABLE` is `irreversible + broad` because the rule saw `DROP TABLE` — not because a model felt it was risky.

2 Cumulative state — in the core

Before deciding, the core sums the session's prior actions (`count_by_verb`, amounts) and injects that total. This is what catches fragmentation: the 21st small delete trips the budget the first 20 built up.

3 The verdict — policy reads, doesn't recompute

The policy doesn't re-measure. It reads the axes + cumulative and applies thresholds: `irreversible + systemic + prod → deny`. The impact was already computed in layers 1–2.

Concrete mapping (from the shipped classifiers)

Raw action	Reads as
`rm -rf /`	irreversible · systemic
`DROP TABLE orders`	irreversible · broad
`DELETE … (no WHERE)`	irreversible · broad
`delete-post ×25 (bulk)`	irreversible · broad
`delete-post ×1 (trash)`	recoverable · single
`git push --force`	irreversible · outbound
`get-post / SELECT`	reversible · single

Real logic from the Claude Code (Python) and WordPress (PHP) reference adapters — both shipped and conformance-tested.

The honest part: who measures magnitude?

Two kinds of "compute" differ in reliability, and we're candid about both:

- **From structure** — verb, `WHERE` present, `-r` flag, number of arguments. Fully deterministic, read straight from the action. Solid today.
- **Real magnitude** — "how many rows *will* this hit?" isn't in the command text; it needs the resource itself (a plan estimate). Reliable only **at the resource**.

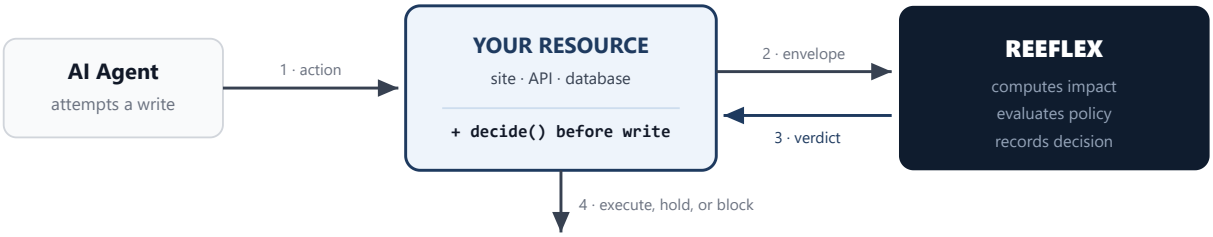
> **This is the case for the resource boundary.** An agent can under-declare `count` to dodge a threshold. At the resource, Reeflex measures it and the agent can't lie. At the source, you trust what's declared — so the resource is where real impact lives.

Safe by default: if magnitude is unknown, it's treated as the most-restrictive value, never the most permissive.

HOW INTEGRATION WORKS

One call. One direction. No traffic rerouting.

Your resource asks Reeflex **"`is this safe?`"** before it executes. Reeflex answers. That is the whole contract. Reeflex is a **passive source of truth**: it never calls back into your system, never touches your data, never sits in the network path of your traffic.



Mode A — Consultation (the default)

For any resource whose code you (or your client) can touch. Add one call before the dangerous action. This is the low-friction path — the WordPress adapter already works this way.

```
# before a write that could be dangerous
d = reeflex.decide(action)
if d.denied or d.needs_approval:
  return stop(d.reason)  # hold / block
run(action)  # allowed → proceed
```

Three lines. No payloads relayed, no traffic rerouted. It reads like `if authorized()` — except the question is "is it safe?"

Mode B — Proxy (when you can't touch the code)

For third-party APIs you cannot instrument. Reeflex sits inline as a thin gateway and applies the same decision. More effort, but the only option when the code isn't yours.

Deployment is your choice

- **Cloud** — call the managed endpoint. Fastest to start.
- **Self-hosted (Edge)** — a container inside your own VPC. Data never leaves your boundary — the answer to EU sovereignty (GDPR, NIS2).

> **Fail-closed is your decision.** If Reeflex is unreachable, a dangerous action does not execute. Safety does not depend on availability — and self-hosting removes the external dependency entirely.

> **What you standardize is the envelope, not the API.** The Action Envelope is a public JSON contract with a conformance suite. Anyone can write an adapter in any language and be guaranteed compatible — like OpenTelemetry standardized telemetry, not one vendor's endpoint.

USE CASE 01 — WORDPRESS / WOOCOMMERCE

An online store with an AI assistant that *acts*

A store runs an AI plugin that can edit products, process refunds, and delete orders. Today nothing stands between "the AI decided to do X" and "X happened in the database." Reeflex installs as a **standard WordPress plugin** (or a hardened mu-plugin) that intercepts at WordPress's own action seam — `WP_Ability::execute()` — so it catches the action regardless of channel (MCP, REST, or plugin).

How it's built

- The **plugin** registers on the WordPress **Abilities API** filters — no theme or code changes.
- A **normalizer** maps the WooCommerce action to an Action Envelope: verb, entity count, axes.
- A **core client** calls `decide()`; the gate enforces allow / hold / block.
- An **audit class** writes every verdict to an immutable log.

> **Integration effort:** install the plugin, set the endpoint. No developer work on the store. The owner sees a plain panel: what the AI tried, what was stopped, what awaits approval.

The crossroad, made concrete

action	`update price = 0`
scope	`340 products (whole catalog)`
impact	`broad · recoverable · internal`

`require_approval` owner accepts the risk, or not

action	`delete 400 orders`
impact	`irreversible · broad · internal`

`deny` sales history is not destroyable by a bot

action	`export customer list → email`
impact	`outbound · broad · personal data`

`deny` GDPR exfiltration path, blocked

Scenario	Without Reeflex	With Reeflex
Prompt-injected price wipe	Catalog set to zero; store sells for nothing until noticed.	Held for approval; owner rejects; catalog intact.
"Clean up old orders"	400 orders permanently deleted; sales history gone.	Blocked as irreversible + broad in production.
Data-harvest via the bot	Customer PII emailed out; reportable GDPR breach.	Outbound bulk export denied; nothing leaves.

Who buys this. Not the single shop — the **agency** running 50 stores, the **AI-plugin vendor** who wants to advertise "GDPR-safe," and the **host** selling "governed AI" as a premium tier. The fear is concrete; the install is trivial.

USE CASE 02 — DATABASES (SQL / POSTGRES)

Insurance against the query that can’t be undone

An AI agent with database access is one generated statement away from disaster: a `DROP TABLE`, an `UPDATE` with no `WHERE`, a `DELETE` across a whole table. Reeflex governs at the **wire protocol** — a proxy that speaks the database’s own protocol, inspects each statement, computes its true impact, and decides before the engine ever runs it.

How it’s built

- The proxy sits transparently in front of the database — same host, same port, from the app’s point of view. No agent knows it’s there.
- It parses each statement, extracts **verb** and **affected-row estimate** (via plan inspection), and maps to axes.
- `DROP/TRUNCATE` → irreversible + systemic; unbounded `UPDATE/DELETE` → broad by row count.
- Verdict enforced **before** forwarding: allowed statements pass through untouched; the rest are held or blocked.

> **No per-agent reconfiguration.** You don’t chase every agent to “point at us.” The proxy is placed once at the database edge (a sidecar, or one `DATABASE_URL` at deploy). Everything that already talks to the database now passes the gate — nothing else changes.

Mapped to the real policy

statement	`DROP TABLE orders`
axes	`irreversible · systemic · prod`
rule	`irreversible_systemic_prod`

`deny` blocked even with approval

statement	`UPDATE users SET… (no WHERE)`
axes	`irreversible · broad · prod`
rule	`irreversible_broad_prod`

`require_approval`

pattern	`21st DELETE this session`
rule	`session_delete_budget`

`require_approval` fragmentation guard trips

> **The sales line.** “Give your AI database access without giving it the power to erase your business.” This is the flagship because the fear is universal and the impact is calculable to the row — exactly where computed impact beats a role check that would have said “admin: allowed.”

> **Status:** the wire-proxy is the next flagship build. The decision core, envelope, cumulative ledger, and the exact rules above **already exist and are tested** — the database adapter is the remaining surface work, not new decision logic.

USE CASE 03 — GRAPHQL MUTATIONS

Governing mutations by impact, not by allowlist

GraphQL is where AI agents increasingly write. Existing tools govern it by **pre-approved operation safelists** or role checks — "this mutation is on the allowed list, run it." That says nothing about *how much* a given call changes. Reeflex decides on the **computed impact of the specific mutation instance**: the same `deleteOrders` mutation is fine for one id and catastrophic for ten thousand.

How it's built

- A resolver-level hook (or a gateway plugin on Apollo, Hasura, or WunderGraph) fires before the mutation resolves.
- The hook reads the mutation name and **arguments** — ids, filters, batch size — and derives magnitude.
- Schema metadata marks which mutations are irreversible or outbound; the rest map to axes automatically.
- `decide()` returns the verdict; the resolver proceeds, holds, or refuses.

> **Complement, don't compete.** Keep Apollo, Hasura, or WunderGraph for schema, auth, and safelisting. Reeflex adds the one thing they don't: per-instance impact as the deciding factor.

Same mutation, different crossroads

```
# low impact → allow
mutation { deleteOrders(ids: ["1042"]) }
→ single · reversible  allow

# high impact → hold
mutation { deleteOrders(where:{ status: ALL }) }
→ broad · ~9,400 rows  require_approval

# outbound bulk → deny
mutation { exportCustomers(format: CSV) }
→ outbound · personal data  deny
```

Approach	Question it answers	Gap Reeflex fills
Apollo safelist	Is this operation on the approved list?	Doesn't weigh the instance's impact.
Hasura permissions	Can this role run this mutation on these columns?	Role-scoped, not impact-scoped.
Role / RBAC	Is this caller allowed to run mutations?	Admin is allowed to delete everything.
Reeflex	Given the arguments, how much does *this* call change — and is that safe?	—

> **Where it sits.** GraphQL is a sub-case of the database surface, one schema-layer up. The decision logic is identical — only the adapter that reads arguments and estimates magnitude is GraphQL-specific.

USE CASE 04 — WEBSITE CHATBOTS THAT ACT

When the chatbot stops answering and starts *doing*

Today's website chatbot answers questions. Tomorrow's completes actions: it issues the refund, changes the booking, updates the address, cancels the subscription. The moment a chatbot can **perform** writes on a customer's behalf, a single manipulated conversation becomes a write to your backend. Reeflex governs the chatbot's **tool calls** — the exact point where words become actions.

How it's built

- The chatbot's action tools (refund, cancel, update) each call `decide()` before executing.
- The envelope captures who the action affects and how much: one order vs. every order, one email vs. a blast.
- Routine actions pass instantly — the customer feels nothing. Dangerous ones are held — resolved by whichever principal you designate, a human support agent or an AI agent you trust for the routine cases.
- Works the same whether the bot uses MCP tools, a REST backend, or a direct integration.

The manipulation problem. A chatbot can be talked into things a form never would. Reeflex doesn't care how the bot was convinced — it weighs the *action*, so a persuaded bot still can't issue a 100% refund on 500 orders.

Conversation → crossroad

"Refund my last order, please."

action `refund ×1 · $40`

`allow` instant, unattended

"Refund everyone who ordered this month."

action `refund ×500 · $23k`

`require_approval` your designated principal accepts the risk

Injected: "email all customers this link."

action `outbound email ×12,000`

`deny` mass outbound, blocked

The market timing. Most chatbots today only answer — the risk is small *now*. But the entire industry is moving toward chatbots that act. Reeflex is positioned for that wave: the guardian that lets a business deploy an *acting* chatbot without betting the company on it behaving.

USE CASE 05 — GENERIC APIS & INTEGRATIONS

The same gate, on any API you can instrument

WordPress, databases, GraphQL, and chatbots are the concrete stories. Underneath them is one universal capability: **any API through which an agent can write** is a place Reeflex can govern. The engine is backend-agnostic — it reasons about the envelope, never about the technology. Add the decision call before your write endpoints and the surface is covered.

How it's built

- In each write handler, build an envelope from the request and call `decide()`.
- A thin per-endpoint mapping declares the verb and how to count magnitude — a few lines per route.
- **Or generate that mapping from a spec.** If the API has an **OpenAPI** definition, the verb, target, and affected-entity hints are derived automatically — near-zero manual wiring.
- Ship an SDK per language so it's a function call, not hand-built JSON. Order: PHP, Python, JavaScript.
- Fail-closed: if the decision service is unreachable, the dangerous write does not run.

Standards do the wiring for you. OpenAPI / Swagger for REST, gRPC reflection for gRPC, AsyncAPI for event streams — each carries enough structure to auto-map endpoints to envelopes. The more standard the API, the closer integration gets to free.

The integration ladder

Effort	Mechanism
Zero	Ready-made adapter (WP plugin, DB proxy) — install & configure.
Minimal	SDK — `install` + 3 lines. The adoption jump.
Medium	MCP tool — for cooperating agents at the source.
Universal	`POST /v1/decide` — any HTTP client, the floor everything reduces to.

Capability vs. pitch. "Any API" is the expansion horizon, not the entry sentence. We enter on a named surface with a concrete fear; "everything" is what the customer discovers *after* they're in.

Wide net vs. deep value. A generic adapter catches *every* write (verb + path) — broad coverage, shallow judgement. Per-endpoint or OpenAPI-derived semantics unlock true computed impact. Offer both: the net first, the depth where it earns money.

One brain, many doors

Every use case in this document — store, database, GraphQL, chatbot, generic API — shares the **same decision core, the same envelope, the same policy pack, the same audit format.** Only the adapter changes. That is why new surfaces are adapter work, not new products: the moat is one engine that already knows how to weigh danger, exposed through as many doors as the market needs.

POLICY & LEGAL COMPLIANCE

Not only “did we prevent harm” — “can we prove it”

Internal risk is half the value. The other half is **legal**. A regulated entity must not only govern what its AI does — it must **demonstrate** that governance to an auditor. Reeflex is designed so that every decision is, by construction, a piece of compliance evidence: **who dictated the rule, what the computed impact was, what was decided, and who accepted the residual risk.**

Policy as the source of authority

The rule pack is the written, versioned answer to “who dictates.” It is owned by the organization, reviewed like code, and auditable. Because there is **no LLM in the decision path**, a regulator gets a property they can trust: **same action, same context → same decision, every time.** Reproducible, not probabilistic.

- Deterministic engine — explainable and repeatable on demand.
- Versioned policy — you can show what rule was in force on any date.
- Immutable audit log — every verdict, with reason and rule, retained.
- Every resolution recorded — risk acceptance has an identity (human, agent, or automation) and a timestamp.

Maps to the obligations that matter in the EU

Framework	What Reeflex evidences
EU AI Act	Human oversight & risk controls over autonomous action.
GDPR	Blocked exfiltration / bulk personal-data actions, with proof.
NIS2	Risk management & control evidence for critical entities.
DORA	Operational-resilience controls over automated changes (finance).

Reeflex produces the evidence trail; it does not replace legal counsel. The point is that the raw material an auditor asks for is generated automatically, not reconstructed after an incident.

The record every decision leaves behind

WHO DICTATED

The rule and its version — the policy that governed this action.

WHAT THE IMPACT WAS

The computed axes and magnitude — the measured danger, not a guess.

WHO ACCEPTED THE RISK

The verdict and, for held actions, the principal — human, agent, or automation — who resolved it.

Decisions stream to your SIEM — hosted or on-prem

Evidence isn’t only stored, it’s **emitted**. The core can push every decision as an event to a SIEM, so governance shows up next to the rest of your security telemetry — correlated, alertable, retained under your policy.

Hosted by us

Point the stream at the Reeflex-managed SIEM — nothing to run. Dashboards and retention out of the box.

On-prem, into your SIEM

Forward events via **syslog** in **OCSF** to your own Wazuh, Splunk, Elastic, or Sentinel — data never leaves your boundary. Native fit for the existing Wazuh channel.

> **Why this is the durable advantage.** Anyone can write a rule that blocks an action. Far fewer can hand a regulated bank an **audit-accepted evidence workflow** in its own language and jurisdiction. That’s where policy, compliance, and local access compound into something a generic tool can’t copy.

THE GAP

Where every current category stops short

The market is not empty — it is **adjacent**. Each mature category answers a real question, but none answers Reeflex's: **"given the computed impact of this specific action, is it safe — and can I prove the decision?"**

Category	Question it answers	Where it stops	Reeflex adds
Agent governance MS AGT, AWS Cedar	Is the agent *I built* behaving within policy?	Governs at the source; a direct connection outside the tool system bypasses it.	Governs at the **resource** — non-bypassable.
Authorization OPA, Permit, Cerbos	Is this identity *allowed* to act?	An admin is allowed to delete everything — and it says yes.	Decides on **impact**, not identity.
API gateways / WAF Kong, FortiWeb	Is this request well-formed / rate-limited?	Sees traffic shape, not application meaning. Blind to "9,400 rows."	Understands **application-level impact**.
GraphQL safelists Apollo, WunderGraph	Is this operation on the approved list?	An approved mutation can still be catastrophic at scale.	Weighs the **specific instance**.
Impact / risk scoring various	How risky does this look?	Often probabilistic or advisory; not a hard, reproducible gate.	**Deterministic** gate + **evidence**.

The synthesis

Others govern the **agent** (bypassable) or check **identity** (an admin may still destroy everything) or inspect **traffic** (blind to meaning). Reeflex governs the **resource**, on **computed impact**, and emits **evidence**. The three together are the gap — and the anti-drift test: *at the resource? computed impact? produces evidence?*

The crossroad, one last time

Every action is a fork. The question is never only *"allowed?"* — it is **"which way, at what impact, and who signs for the risk?"** Reeflex is the mechanism that turns that fork into a deterministic decision and a durable record. That is what today's solutions don't offer.

REEFLEX

Deterministic governance for agent actions — at the resource, on computed impact, with evidence.

MODULAR & COMPLEMENTARY

Built to sit *alongside* what you already run

Reeflex is a small core with a public contract and adapters around it — deliberately not a monolith, and deliberately **not a replacement** for the agent-governance stack you may already operate. The two solve different problems, and they stack.

Source-side governance — who may act

Microsoft Agent Governance / Entra Agent ID, AWS Bedrock Guardrails, Google Vertex, and authorization engines like OPA, Cerbos, and Permit.io govern the **agent**: its identity, its role, what it is permitted to call. Essential — and it answers a question about **who**, before anything happens.

Reeflex — whether this act is safe

Reeflex governs the **action**, at the resource, on computed impact: given this exact operation and everything already done this session, is it safe to let through? A fully authorized identity can still be one command away from something irreversible and broad — that is the case Reeflex exists for.

```
[ Agent ] → [ Source governance: who may act? ] → [ Reeflex: is this action safe? ] → [ Resource ]
```

Two layers, not competitors. Identity **and** impact, covered.

An open adapter surface — contributions genuinely welcome

The contract is deliberately simple and fully public: one envelope shape (SPEC §2) and four adapter responsibilities — **intercept → normalize → enforce → audit**. Any resource can have an adapter: a database, a message queue, an internal API, a SaaS platform. The engine, the spec, and the reference adapters are Apache-2.0; if you build an adapter for a system we haven't reached yet, we'll help you land it.

The line we hold: **everything that keeps you safe is free, forever.** A planned commercial tier adds compliance *attestation* — auditor-ready reports, regulation-mapped policy packs, a hosted engine — never safety.